

StatsPAI: A Unified, Agent-Native Python Toolkit for Causal Inference and Applied Econometrics

Biaoyue Wang ^{1,2¶} and Scott Rozelle^{1,2}

1 Rural Education Action Program, Stanford Center on China's Economy and Institutions, Stanford University, United States  2 StatsPAI Inc., United States  Corresponding author

DOI: 10.xxxxxx/draft

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: Richard Liu

Reviewers:

- @Carol-seven
- @FATelarico
- @MartinSpindler

Submitted: 07 April 2026

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

Summary

StatsPAI is an open-source Python package for causal inference and applied econometrics. It gives empirical researchers a single interface for estimating, diagnosing, comparing, and reporting models that are otherwise spread across many specialized packages or proprietary statistical environments. A single `import statspai as sp` reaches estimators for the main families of applied work: regression and panel models, instrumental variables, the modern difference-in-differences and regression-discontinuity toolkits, synthetic control and matching, and machine-learning estimators of heterogeneous treatment effects. It also covers the diagnostics, robustness checks, and reporting that surround them. The full catalogue of more than 1,100 registered functions across more than 80 submodules is enumerated in the package documentation rather than here.

Results from the mature estimators share a common reporting surface, so the same calls produce a summary, a figure, a LaTeX or Word table, or a citation. StatsPAI is also agent-native: every registered function exposes a machine-readable schema, a structured description of its arguments and outputs that a program can parse directly, together with structured failure metadata. Such schemas let LLM-driven research assistants discover estimators, choose among alternatives, and surface a method’s assumptions without parsing free-form prose, the capability that most distinguishes StatsPAI from a conventional estimator library. The source code is available at <https://github.com/brycewang-stanford/StatsPAI> and archived on Zenodo (Wang & Rozelle, 2026).

Statement of Need

Applied researchers face a fragmented software landscape. Stata offers an integrated workflow, but it is proprietary and does not expose a typed, machine-readable interface for AI-assisted analysis. R provides excellent method-specific packages such as *did* (Callaway & Sant'Anna, 2021), *rdrobust* (Calonico et al., 2014), *Synth* (Abadie et al., 2010), *grf* (Athey et al., 2019), and *lme4* (Bates et al., 2015), but these packages use different APIs, object systems, and output conventions. Python has strong pieces of the causal inference ecosystem, including DoWhy for graphical causal models (Sharma & Kiciman, 2020), EconML for machine-learning treatment effect estimation (Battocchi et al., 2019), CausalML for uplift modeling (Chen et al., 2020), and DoubleML for double/debiased machine learning (Bach et al., 2022). None of these tools, however, is intended to cover the full applied-econometrics workflow from design diagnosis through estimation, robustness, and publication output.

StatsPAI addresses this gap for graduate students, applied economists, policy researchers, and data scientists who want a Python-native workflow without giving up the breadth of Stata or the methodological depth of R. Its goal is not to replace every specialized implementation, but to provide a coherent empirical workspace: shared formula conventions, compatible result surfaces

for mature estimators, export methods where supported, citations attached to estimators, and validation metadata that make the relationship between methods, assumptions, and evidence explicit.

State of the Field

Existing Python packages are strongest when they focus on a narrower problem: DoWhy emphasizes identification, graphical assumptions, and refutation; EconML and CausalML focus on heterogeneous effects and uplift modeling; DoubleML implements orthogonal-score estimators for a well-defined family of double machine-learning designs. These packages are complementary to StatsPAI, and several of its ideas follow the same methodological literature, including double/debiased machine learning (Chernozhukov et al., 2018), causal forests (Wager & Athey, 2018), and meta-learners (Künzel et al., 2019). In the high-dimensional and double machine-learning tradition specifically, StatsPAI builds on established reference implementations rather than around them: it provides a faithful Python port of the rigorous (data-driven) Lasso and post-double-selection estimators of the hdm package (Alexandre Belloni et al., 2012; A. Belloni et al., 2014; Chernozhukov et al., 2016), validated for numerical agreement with hdm, and orthogonal-score estimators aligned with the DoubleML implementations in Python and R (Bach et al., 2022, 2024). Researchers can thus reproduce the high-dimensional econometrics workflow inside the same agent-addressable interface used for the rest of the package.

The general-purpose regression toolkits applied economists already rely on, such as statsmodels and linearmodels in Python or fixest in R, sit at a different layer, supplying core regression machinery that StatsPAI builds on rather than competes with. Its contribution is therefore the integration layer itself, not a single better estimator: contributing one more estimator to each existing project would still leave users with incompatible result classes, separate diagnostic conventions, and no unified agent-facing schema. StatsPAI earns a standalone existence by being the layer that makes a heterogeneous collection of estimators (classical and machine-learning, Python-native and R/Stata-aligned) behave as one coherent, agent-addressable workspace, with broad method coverage, shared reporting, explicit citations, stable/experimental API metadata, and cross-language parity checks.

Software Design

StatsPAI is organized around method families and a registry layer. Researchers can call focused functions, such as an IV or regression-discontinuity estimator, or use higher-level dispatchers that select among variants within a design family. The registry records function names, parameters, examples, stability tiers, limitations, citations, and schema information, making the package usable both from a notebook and from external systems such as a Model Context Protocol server. In practice an assistant can query the registry for estimators valid for a detected design, invoke one, read back structured diagnostics and assumption violations, and decide the next step through typed schemas rather than free-form prompts.

The central design choice is a shared result interface: estimators return structured objects that store coefficients, uncertainty estimates, diagnostics, fitted values, plots, and exporter hooks in predictable locations. This lowers the switching cost between classical econometric and modern machine-learning estimators, and makes validation easier because tests can compare common fields across implementations.

The package is implemented mainly in Python on top of NumPy, SciPy, Pandas, statsmodels, scikit-learn, and linearmodels, and supports Python 3.9 and newer. Optional accelerator backends are used only where they materially change the computation: PyTorch for neural causal estimators, JAX for selected bootstrap and linear-algebra workloads, and a Rust/PyO3 kernel for high-dimensional fixed-effect and cluster-variance routines. It is distributed via PyPI under the MIT license.

90 These choices carry costs worth stating plainly. A shared result interface means adding or
 91 upgrading an estimator is never purely local, and the reporting and export hooks are guaranteed
 92 only for the mature estimators, which is why auxiliary helpers advertise narrower capabilities
 93 through the registry rather than claiming a uniformity they do not have. Favouring breadth
 94 also trades against depth: for any single design, a dedicated package may expose more
 95 edge-case options than StatsPAI does today. Performance is likewise not the first priority
 96 of the default install, which runs on a pure-Python NumPy/SciPy stack and reaches for the
 97 PyTorch, JAX, or Rust backends only when present, holding those accelerated paths to the
 98 same documented numerical tolerances as the fallbacks. The Rust/PyO3 kernel in particular
 99 carries a compiled-language build cost: platform-specific wheels or a Rust toolchain, plus a
 100 separately maintained crate. We contain this by keeping the kernel optional, with transparent
 101 pure-Python fallback. We accept these costs deliberately: for researchers who must compare
 102 estimators, switch designs, and produce reproducible output within one project, a single
 103 coherent, agent-addressable interface outweighs the loss of per-method specialization.

104 Research Impact Statement

105 StatsPAI ships a concrete validation and community-readiness dossier built from two
 106 complementary tracks. The first is a cross-language parity harness: StatsPAI, R, and Stata
 107 are run on the same input bytes and their numerical output compared directly. The harness
 108 checks more than sixty estimator modules against a reference R implementation, the large
 109 majority also against Stata. Closed-form estimators agree to machine precision; iterative
 110 and machine-learning estimators agree within pre-registered, documented tolerances, and
 111 the few remaining convention gaps are disclosed rather than hidden. Within this harness,
 112 the high-dimensional methods reproduce the published worked examples of the *hdm* package
 113 (Chernozhukov et al., 2016) (its growth-convergence, institutions-and-development, and
 114 gender-wage-gap applications), and the double-machine-learning estimators are checked
 115 against DoubleML (Bach et al., 2022, 2024) on shared data, an independent check against
 116 those established reference implementations. The second track ships canonical teaching
 117 datasets in *sp.datasets*. Where the original extract is redistributable it is bundled as real
 118 data, so that analyses run on the study data rather than on a stand-in: returns-to-schooling
 119 IV (Card), job-training (LaLonde/Dehejia-Wahba), RD elections (Lee), synthetic control
 120 (Abadie-Diamond-Hainmueller), and four worked examples from Cunningham's *Causal*
 121 *Inference: The Mixtape* (Cunningham, 2021). For the Mixtape examples the point estimates —
 122 and, where a reference implementation exists, the standard errors — agree with Stata 18 and
 123 the R *did* package to the tolerances recorded in the repository's parity tests. The remaining
 124 entries, including the multi-period difference-in-differences panel (Callaway-Sant'Anna), are
 125 deterministic simulated replicas calibrated so that the canonical estimator recovers values near
 126 the published result; for those, exact numerical replication is deliberately not claimed. The suite
 127 also includes a 1000-replication coverage run for representative OLS, difference-in-differences,
 128 and strong-instrument IV designs, with empirical coverage close to the nominal 95 percent
 129 level. A reviewer-facing validation dossier and short reviewer guide ship with the repository
 130 documentation.

131 The near-term impact is a more reproducible workflow for applied policy evaluation: sharing
 132 one interface, researchers can compare estimators on the same data, export tables with the
 133 same metadata, and record the citations and assumptions attached to each analysis. StatsPAI
 134 is currently being used in an ongoing working paper connected to the Rural Education Action
 135 Program at Stanford University, *Family contagion of screen time? Within-person evidence from*
 136 *six waves in China* (Wang, Zhang, and Hou, in preparation), which relies on the package for its
 137 panel and within-person estimation; no peer-reviewed research article using the package has yet
 138 been published. The impact claim therefore rests on three things a reviewer can check directly:
 139 active use in an ongoing working paper, public distribution on PyPI, and the reproducible
 140 validation materials and worked examples bundled with the repository.

141 AI Usage Disclosure

142 Generative AI tools, including Claude Code and OpenAI ChatGPT/Codex, were used for code-
143 generation assistance, refactoring suggestions, test scaffolding, documentation drafting, and
144 manuscript copy-editing. Exact model identifiers were not retained for all exploratory sessions.
145 Human authors made the core design decisions; reviewed, edited, and checked AI-assisted code
146 and prose; and checked citations and software claims against repository evidence. The authors
147 will not use generative AI to produce substantive responses to journal editors or reviewers.
148 All authors take responsibility for the correctness, originality, licensing, and compliance of the
149 package and this paper.

150 Author Contributions

151 **Biaoyue Wang** conceived and designed the package, implemented the estimators, registry,
152 schema layer, and result objects, wrote the documentation, tests, and validation suites, and led
153 the drafting of this paper. **Scott Rozelle** provided guidance on the package's design direction
154 and target research workflows, and contributed to the writing, review, and revision of this
155 paper. Both authors reviewed and approved the final manuscript and take responsibility for
156 the correctness of the package and this paper.

157 Acknowledgements

158 The authors thank the Stanford Rural Education Action Program (REAP) research community
159 and the CoPaper.AI team for feedback on early workflows. StatsPAI Inc. is the legal entity
160 associated with the project, and CoPaper.AI is a commercial downstream product that may
161 call the MIT-licensed StatsPAI package; the StatsPAI package itself is permanently open
162 source under the MIT license. The authors are also grateful to the developers of NumPy, SciPy,
163 Pandas, statsmodels, scikit-learn, linearmodels, PyTorch, JAX, and the broader open-source
164 scientific Python ecosystem that StatsPAI builds upon.

165 References

- 166 Abadie, A., Diamond, A., & Hainmueller, J. (2010). Synthetic control methods for comparative
167 case studies: Estimating the effect of California's tobacco control program. *Journal of the*
168 *American Statistical Association*, 105(490), 493–505. [https://doi.org/10.1198/jasa.2009.](https://doi.org/10.1198/jasa.2009.ap08746)
169 [ap08746](https://doi.org/10.1198/jasa.2009.ap08746)
- 170 Athey, S., Tibshirani, J., & Wager, S. (2019). Generalized random forests. *The Annals of*
171 *Statistics*, 47(2), 1148–1178. <https://doi.org/10.1214/18-aos1709>
- 172 Bach, P., Chernozhukov, V., Kurz, M. S., & Spindler, M. (2022). DoubleML – an object-
173 oriented implementation of double machine learning in Python. *Journal of Machine Learning*
174 *Research*, 23(53), 1–6. <https://jmlr.org/papers/v23/21-0862.html>
- 175 Bach, P., Kurz, M. S., Chernozhukov, V., Spindler, M., & Klaassen, S. (2024). DoubleML:
176 An object-oriented implementation of double machine learning in R. *Journal of Statistical*
177 *Software*, 108(3), 1–56. <https://doi.org/10.18637/jss.v108.i03>
- 178 Bates, D., Mächler, M., Bolker, B., & Walker, S. (2015). Fitting linear mixed-effects models
179 using lme4. *Journal of Statistical Software*, 67(1), 1–48. [https://doi.org/10.18637/jss.](https://doi.org/10.18637/jss.v067.i01)
180 [v067.i01](https://doi.org/10.18637/jss.v067.i01)
- 181 Battocchi, K., Dillon, E., Hei, M., Lewis, G., Oka, P., Oprescu, M., & Syrgkanis, V. (2019).
182 *EconML: A Python package for ML-based heterogeneous treatment effects estimation.*
183 <https://github.com/py-why/EconML>

- 184 Belloni, Alexandre, Chen, D., Chernozhukov, V., & Hansen, C. (2012). Sparse models and
185 methods for optimal instruments with an application to eminent domain. *Econometrica*,
186 80(6), 2369–2429. <https://doi.org/10.3982/ECTA9626>
- 187 Belloni, A., Chernozhukov, V., & Hansen, C. (2014). Inference on treatment effects after
188 selection among high-dimensional controls. *The Review of Economic Studies*. <https://doi.org/10.1093/restud/rdt044>
- 190 Callaway, B., & Sant'Anna, P. H. C. (2021). Difference-in-differences with multiple time periods.
191 *Journal of Econometrics*, 225(2), 200–230. <https://doi.org/10.1016/j.jeconom.2020.12.001>
- 192 Calonico, S., Cattaneo, M. D., & Titiunik, R. (2014). Robust nonparametric confidence
193 intervals for regression-discontinuity designs. *Econometrica*, 82(6), 2295–2326. <https://doi.org/10.3982/ECTA11757>
- 195 Chen, H., Harinen, T., Lee, J.-Y., Yung, M., & Zhao, Z. (2020). *CausalML: Python package*
196 *for causal machine learning*. <https://doi.org/10.48550/arXiv.2002.11631>
- 197 Chernozhukov, V., Chetverikov, D., Demirer, M., Duflo, E., Hansen, C., Newey, W., & Robins,
198 J. (2018). Double/debiased machine learning for treatment and structural parameters. *The*
199 *Econometrics Journal*, 21(1), C1–C68. <https://doi.org/10.1111/ectj.12097>
- 200 Chernozhukov, V., Hansen, C., & Spindler, M. (2016). hdm: High-dimensional metrics. *The*
201 *R Journal*, 8(2), 185–199. <https://doi.org/10.32614/RJ-2016-040>
- 202 Cunningham, S. (2021). *Causal inference: The mixtape*. Yale University Press. ISBN: 978-0-
203 300-25168-5
- 204 Künzel, S. R., Sekhon, J. S., Bickel, P. J., & Yu, B. (2019). Metalearners for estimating
205 heterogeneous treatment effects using machine learning. *Proceedings of the National*
206 *Academy of Sciences*, 116(10), 4156–4165. <https://doi.org/10.1073/pnas.1804597116>
- 207 Sharma, A., & Kiciman, E. (2020). *DoWhy: An end-to-end library for causal inference*.
208 <https://doi.org/10.48550/arXiv.2011.04216>
- 209 Wager, S., & Athey, S. (2018). Estimation and inference of heterogeneous treatment effects
210 using random forests. *Journal of the American Statistical Association*, 113(523), 1228–1242.
211 <https://doi.org/10.1080/01621459.2017.1319839>
- 212 Wang, B., & Rozelle, S. (2026). *StatsPAI: The agent-native causal inference and econometrics*
213 *toolkit for python* (Version 1.15.6). <https://doi.org/10.5281/zenodo.19933900>